Introduzione a

Python

Tratti salienti

- Inventato da Guido van Rossum nel ~1990
- Ricco ecosistema di librerie e framework

- Linguaggio imperativo orientato agli oggetti
- Interpretato (prevalentemente)
- Dinamicamente tipizzato (prevalentemente)
- Offre *garbage collection*

Elementi di sintassi

```
n = int(input('Type a number, and its factorial will be printed: '))

if n < 0:
    raise ValueError('You must enter a non-negative integer')

factorial = 1
for i in range(2, n + 1):
    factorial *= i
print(factorial)
```

Osservazioni:

Sintassi snella

Non ci sono dichiarazioni di tipi (tipico dei ling. di scripting)

L'indentazione è rilevante

Elementi di sintassi

Supporta *funzioni annidate*

Scoping statico

```
def f(a):
    b = 2*a
    def g(c):
        print("a={}, b={}, c={}".format(a,b,c))
    g(10)

f(3)
g(4)
```

Funzione annidata (nested)

g ha accesso alle variabili locali di f

Errore a runtime: “NameError: name 'g' is not defined”
g non è visible fuori da f

Output:

```
a=3, b=6, c=10
```

Esecuzione

```
python <nomefile>
```

Oppure (in ambiente Unix):

iniziare il programma sorgente con:

```
#!/bin/python
```

Oppure invocare l'interprete come *read-eval-print loop* (REPL):

`python` oppure `python3`

Numerose implementazioni alternative

Esempio: *mypy*, offre type checking statico

Elementi di semantica

- **Tutti i valori sono oggetti**
- **Tutte le variabili sono riferimenti**
- Come se in Java non esistessero i tipi primitivi, ma solo i wrapper

- Funzione **`id(<exp>)`**: restituisce l'indirizzo dell'oggetto puntato da <exp>
  - Ovvero, il valore del riferimento

- Funzione **`type(<exp>)`**: restituisce il tipo dell'oggetto puntato da <exp>

Modello dei dati e Data Objects

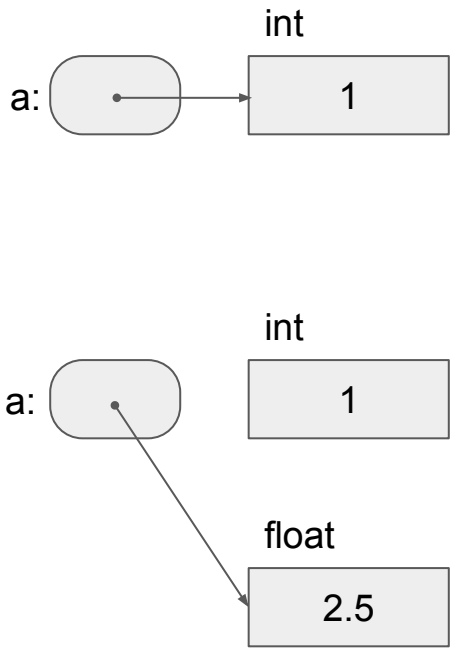
(Locazione, Nome, Valore, Tipo)

Codice Python:

```
a = 1
id(a)   → 140335477971640
type(a) → type 'int'
```

```
a = 2.5
id(a)   → 573821909931
type(a) → type 'float'
```

Memory layout:



Data Objects:

(loc1, a, loc2, “riferimento a int”)
(loc2, anonimo, 1, int)

loc2 = 140335477971640

(loc1, a, loc3, “riferimento a float”)
(loc3, anonimo, 2.5, float)

loc3 = 573821909931

Sistema dei tipi

Ricca varietà di tipi nativi:

- boolean
- *Numerici:* int, float, **complex**
- *Stringhe:* str, bytes, bytearray
- *Collezioni:* list, tuple, dict, set, frozenset

L’utente può creare nuove classi e funzioni

Le funzioni/metodi sono *first-class objects* (si possono passare come argomenti)

Collezioni *built-in*

- **list:** sequenza di oggetti
  - `lista = [1, 2, 3]`
- **tuple:** tupla/n-upla (“ennupla”), generalizzazione di coppia
  - `tripla = (1, “due”, 3.0)`
- **dict:** dizionario, array associativo, associazione chiavi-valori, mappa (Java)
  - `diz = { “pippo”: 1000, “pluto”: 2000 }`
- **set:** insieme matematico (non ordinato e senza duplicati)
- **frozenset:** versione immutabile di set

Esercizio: Qual è la differenza tra list e tuple?

A proposito di interi

Il tipo *int* rappresenta interi di *dimensione arbitraria*

```
>>> 2**50
1125899906842624
```

Esempio di scelta di *astrazione maggiore* rispetto a C, Java, etc.

Più flessibile, meno efficiente

- In Java: classe standard BigInteger
- In C e C++: librerie esterne

Per dettagli, si veda Wikipedia: List of arbitrary-precision arithmetic software

Mutabile o immutabile?

Mutabile vuol dire: cambia il valore, non cambia l'indirizzo (locazione)

Esercizi:

1. Scoprire empiricamente se il tipo **int** è mutabile
2. Scoprire empiricamente se il tipo **list** è mutabile

Type checking

I tipi non sono *dichiarati*, ma esistono e sono *controllati* a tempo di esecuzione

Esempi:

```
>>> 1+"a"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
>>> a=1
>>> a.get(2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'int' object has no attribute 'get'
```

Type checking

Solo *se e quando* il codice viene eseguito

Esempio:

```
a=1
if a==0:
    b=1+”ciao”
```

Questo frammento viene eseguito senza problemi